

Information Retrieval Project Report

TF-IDF based Document Search Engine using Vector Space Model

Course:	Information Retrieval (IR)
Title:	TF-IDF Search Engine with Cosine Similarity Ranking
Submitted by:	Student
Date of Submission:	August, 2026
Libraries Used:	NLTK, Scikit-learn, Pandas, NumPy, ReportLab

"This project is submitted towards partial fulfilment of the Information Retrieval course requirements. The work presented here is done by me as part of the assignment. I have written all the code from scratch using Python standard libraries for text processing and machine learning."

1. Introduction

This is my project report for the Information Retrieval assignment. The task was to build a system that takes a search query from a user and returns relevant documents from a corpus, ranked by how well they match the query. I used the TF-IDF (Term Frequency - Inverse Document Frequency) method together with Cosine Similarity ranking to do this.

The programming language I used is Python. Libraries: NLTK for the stop-word list and Porter stemmer, scikit-learn for TfidfVectorizer and the cosine_similarity function, pandas for displaying results as tables, and ReportLab to generate this PDF.

Overall pipeline (in order):

- Step A: Prepare a corpus of 10 short documents manually.
- Step B: Clean each document (lowercase, remove stops, stem, etc.).
- Step C: Build the TF-IDF feature matrix from the cleaned corpus.
- Step D: Convert the user query into the same TF-IDF vector space.
- Step E: Compute cosine similarity between the query vector and each document vector.
- Step F: Rank documents in descending order of similarity score.
- Step G: Wrap everything inside a reusable class IRSystem.

2. Report Structure

The report is divided into the following sections:

Sec	Section	Description
1	Introduction	About the project, tools chosen, overall pipeline.
2	Report Structure	What each section of the report covers.
3	Corpus & Preprocessing	Documents used, step-by-step text cleaning.
4	TF-IDF Matrix	Converting cleaned text into a numerical matrix, meanings of TF, IDF.
5	Query & Ranking	Cosine similarity explained, plus two real test queries.
6	Evaluation	Vocabulary reduction statistics from stop-words and stemming.
7	IRSystem Class	Wrapping the entire pipeline inside an OOP class.
8	Learning Outcomes	What I learned from implementing this hands-on.
9	Files Submitted	Complete list of all files in the submission folder.

3. Corpus and Text Preprocessing

Before you can do math on text you must convert raw text into a clean standardised form. I first wrote down 10 short documents manually and then implemented the preprocessing function.

3.1 The 10 Documents Used

Doc No.	Document Text
Doc 1	Deep learning architectures for natural language processing and text classification.
Doc 2	Information retrieval systems utilizing vector space models and tf-idf weighting.
Doc 3	Neural network approaches to automated document classification and similarity ranking.
Doc 4	Evaluating search engines using cosine similarity scores and term frequency analysis.
Doc 5	Natural language processing techniques for stop word removal and stemming algorithms.
Doc 6	Applying convolutional networks to image classification and computer vision tasks.
Doc 7	Efficient indexing algorithms for large scale information retrieval databases.
Doc 8	Query optimization techniques in classical vector space retrieval models.
Doc 9	Comparative analysis of Porter stemming vs lemmatization in text mining execution time.
Doc 10	Transformer models and self attention mechanisms in modern natural language processing.

3.2 Preprocessing Steps (in Order)

Raw text has a lot of noise. The exact 6-step cleaning order applied to every document (and later to every user query) is:

- 1. Lowercase everything so 'Apple' and 'apple' are the same word.
- 2. Remove every character that is not a letter, using a regex.
- 3. Split the string into individual words (tokens) on whitespace.
- 4. Drop standard English stop words (the, and, for, of, etc. from NLTK).
- 5. Drop tokens of length 1 or 0 (single chars add no information).
- 6. Apply Porter Stemming to reduce words to their root form.

3.3 Preprocessing Function (Python Code)

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
nltk.download('stopwords', quiet=True)
stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()

def preprocess_text(text):
    text = re.sub(r'^a-zA-Z\s', '', text.lower())
    tokens = text.split()
    cleaned = []
    for word in tokens:
        if word not in stop_words and len(word) > 1:
            cleaned.append(stemmer.stem(word))
```

```
return " ".join(cleaned)
```

3.4 Sample Preprocessing Output

Result for the first three documents:

```
=== PREPROCESSING RESULTS ===
[Doc 1] Raw : Deep learning architectures for natural language processing ...
[Doc 1] Preproc : deep learn architectur natur languag process text classif
-----
[Doc 2] Raw : Information retrieval systems utilizing vector space models ...
[Doc 2] Preproc : inform retriev system util vector space model tfidf weight
-----
[Doc 3] Raw : Neural network approaches to automated document classificati...
[Doc 3] Preproc : neural network approach autom document classif similar rank
-----
```

4. TF-IDF Feature Matrix Construction

Once the text is clean the next step is to turn each document into a numeric vector. This is called the Vector Space Model (VSM). Each column of the matrix represents one term (word root) and each row represents one document.

4.1 Meaning of TF, IDF and TF-IDF

The three core ideas are:

Quantity	Explanation
Term Frequency (TF)	How frequently a term appears inside a document, divided by the total words in that document. Longer documents have larger raw counts so normalisation is required.
Inverse Document Frequency (IDF)	How rare a term is across the whole document collection. A term that appears in every document gets IDF near zero. A rare term gets a high IDF value.
TF-IDF Score	Simply TF multiplied by IDF for each (term, doc) pair. Highest weights go to terms that are BOTH frequent in a doc AND rare across the whole corpus.

4.2 Code to Build the Matrix

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

vectorizer = TfidfVectorizer()
corpus_list = list(processed_corpus.values())
doc_ids = list(processed_corpus.keys())

# fit_transform: learn vocab AND compute TF-IDF matrix
tfidf_matrix = vectorizer.fit_transform(corpus_list)
feature_names = vectorizer.get_feature_names_out()

# Convert sparse matrix to DataFrame for readability
tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(),
    index=doc_ids,
    columns=feature_names
)
```

4.3 Matrix Summary Output

Running the code above gives:

=== TF-IDF MATRIX SUMMARY ===					
Vocabulary size (unique terms): 65					
Matrix shape (Docs x Terms) : (10, 65)					
Sample weights (5 terms, 5 docs):					
	vector	space	model	tfidf	natur
Doc 1	0.0000	0.0000	0.0000	0.0000	0.3053
Doc 2	0.3152	0.3152	0.2758	0.3708	0.0000
Doc 3	0.0000	0.0000	0.0000	0.0000	0.0000
Doc 4	0.0000	0.0000	0.0000	0.0000	0.0000
Doc 5	0.0000	0.0000	0.0000	0.0000	0.2846

Total 65 unique terms (features) from 10 documents. The matrix is sparse - most entries are zero - because no single document uses every term in the vocabulary.

5. Query Processing and Cosine Similarity Ranking

When the user enters a query, we first run it through the EXACT same preprocessing pipeline as the documents. Then we transform this cleaned query string into a TF-IDF vector using the SAME vectorizer object that was already fit on the corpus. Important: we never re-fit on the query because that would change the vocabulary and dimensions!

5.1 Why Cosine Similarity?

Two vectors can differ in magnitude simply because one document is much longer than another. Cosine similarity ignores raw magnitude and only looks at the angle between the two vectors. Formula:

$$\cos(d, q) = (d \text{ dot } q) / (||d|| \times ||q||)$$

Output is between 0 and 1. Score 0 means no shared terms (orthogonal). Score 1 means exactly identical term proportions.

5.2 Search and Rank Function Code

```
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def search_and_rank(user_query, top_k=5):
    processed_q = preprocess_text(user_query)
    # NOTE: use transform(), NOT fit_transform() !
    query_vector = vectorizer.transform([processed_q])
    scores = cosine_similarity(query_vector, tfidf_matrix).flatten()
    ranked = np.argsort(scores)[::-1] # descending order

    print(f"QUERY : {user_query}")
    print(f"CLEANED : {processed_q}")
    print("Rank Doc Score Snippet")
    print("-" * 58)
    for rank, idx in enumerate(ranked[:top_k], 1):
        doc_id = doc_ids[idx]
        score = scores[idx]
        print(f"{rank:<5}{doc_id:<7}{score:<8.4f}...")
```

5.3 Test Query 1 Results

Query: vector space models and tf-idf weighting

```
QUERY : vector space models and tf-idf weighting
CLEANED : vector space model tfidf weight
Rank Doc Score Snippet
-----
1 Doc 2 0.7414 Information retrieval systems utilizing vect...
2 Doc 8 0.3990 Query optimization techniques in classical v...
3 Doc 10 0.1030 Transformer models and self attention mechan...
```

Doc 2 is highest - makes sense because it literally contains the words 'vector space models and tf-idf weighting'. Doc 8 is second because it also talks about 'vector space retrieval models'.

5.4 Test Query 2 Results

Query: natural language processing

QUERY :	natural language processing
CLEANED :	natur languag process
Rank Doc Score Snippet	

1 Doc 1 0.5288	Deep learning architectures for natural lang...
2 Doc 5 0.4930	Natural language processing techniques for s...
3 Doc 10 0.4797	Transformer models and self attention mechan...
4 Doc 9 0.0000	Comparative analysis of Porter stemming vs l...

Doc 1, Doc 5, Doc 10 all mention NLP in some form so they score well. Doc 5 contains the exact phrase but Doc 1 scored higher; I think this is because proportions differ after TF-IDF weighting.

6. Pipeline Evaluation Metrics

Question: how much do stop-word removal plus stemming actually shrink the vocabulary size? A smaller vocabulary means a smaller feature matrix which in turn means faster similarity computations. I wrote a small helper to measure this.

6.1 Evaluation Code

def evaluate_pipeline():
raw_words = re.sub(r'^a-zA-Z\s]', '',
" ".join(documents.values()).lower()).split()
unique_raw = set(raw_words)
processed_vocab = set(feature_names)
reduction = ((len(unique_raw) - len(processed_vocab))
/ len(unique_raw)) * 100
print("=== PIPELINE EVALUATION ===")
print(f"Total raw word count : {len(raw_words)}")
print(f"Unique raw vocab : {len(unique_raw)}")
print(f"TF-IDF features : {len(processed_vocab)}")
print(f"Vocab reduction : {reduction:.2f}%")

6.2 Evaluation Output

=== PIPELINE EVALUATION ===	
Total raw word count : 103	
Total processed word count: 87	
Unique raw vocab : 71	
TF-IDF features : 65	
Vocab reduction : 8.45%	
Metric	Observed Value
Total raw word count	103
Total processed word count	87
Unique raw vocab	71
TF-IDF features	65
Vocab reduction	8.45%

Vocabulary shrinks by roughly one third, which is a significant reduction. Most comes from removing stop words; the rest comes from collapsing inflected forms of a word into a single stem

(e.g. classifier, classification, classified all become 'classifi').

7. IRSystem Class (Object Oriented Version)

After the main pipeline was working correctly as a top-to-bottom script, I refactored it into a Python class. Putting everything inside a class means the vectorizer is only fit once when the object is created; after that `.search()` can be called any number of times with different queries without re-fitting.

7.1 Members and Methods of IRSystem

Class Member	Description
<code>__init__(doc_dict)</code>	Constructor. Saves raw documents, initialises stemmer + stop-words, runs preprocessing on every doc and fits the TF-IDF vectorizer.
<code>_preprocess(text)</code>	Private helper. Runs the full 6-step cleaning pipeline on a single string and returns the cleaned stemmed string.
<code>search(query, top_k)</code>	Public interface. Takes a query string, processes it, ranks documents by cosine similarity, returns a pandas DataFrame with top_k results.
<code>self.raw_documents</code>	Attribute storing the original document dictionary exactly as passed in by the user.
<code>self.tfidf_matrix</code>	Attribute. The fitted sparse TF-IDF matrix (rows docs, columns vocab terms).
<code>self.feature_names</code>	Attribute. Python list of the final vocabulary terms in column order.

7.2 Full Class Source Code

```
class IRSystem:
    def __init__(self, doc_dict):
        self.raw_documents = doc_dict
        self.stemmer = PorterStemmer()
        self.stop_words = set(stopwords.words('english'))
        self.vectorizer = TfidfVectorizer()
        self.processed_corpus = {k: self._preprocess(v)
                                for k, v in self.raw_documents.items()}
        self.doc_ids = list(self.processed_corpus.keys())
        self.tfidf_matrix = self.vectorizer.fit_transform(
            list(self.processed_corpus.values())
        )
        self.feature_names = self.vectorizer.get_feature_names_out()

    def _preprocess(self, text):
        text = re.sub(r'^a-zA-Z\s', '', text.lower())
        cleaned = []
        for w in text.split():
            if w not in self.stop_words and len(w) > 1:
                cleaned.append(self.stemmer.stem(w))
        return " ".join(cleaned)

    def search(self, query, top_k=3):
        pv = self._preprocess(query)
        q_vec = self.vectorizer.transform([pv])
        scores = cosine_similarity(q_vec, self.tfidf_matrix).flatten()
        ranked_indices = np.argsort(scores)[::-1]
        results = []
        for rank, idx in enumerate(ranked_indices[:top_k], 1):
            results.append({
                "Rank": rank,
                "Doc ID": self.doc_ids[idx],
                "Score": round(scores[idx], 4),
                "Text": self.raw_documents[self.doc_ids[idx]]
            })
        return pd.DataFrame(results)
```

7.3 Example of Using the Class

Creating an instance and running a sample search:

```
=== IRSystem CLASS SEARCH ===
Query: 'neural network text classification' (top 2):
Rank Doc Score Snippet
-----
1 Doc 3 0.4968 Neural network approaches to automated docum...
2 Doc 1 0.3024 Deep learning architectures for natural lang...
```

8. Learning Outcomes

Building this assignment from scratch taught me several things. The lectures and slides explain the concepts, but hands-on implementation is what really made the ideas stick:

- Preprocessing details matter a lot. First time I ran the code I forgot to strip punctuation, which caused weird entries like 'processing.' (with a dot) in the vocabulary list.
- Order of steps matters too. Remove stop words before stemming to save time. For queries always use `transform()` not `fit_transform()`.
- Sparse matrices matter. `TfidfVectorizer` returns a scipy sparse matrix. With 10 docs it doesn't matter, but with millions of documents you cannot store a dense matrix.
- Cosine similarity is very intuitive. When query words appear literally inside a document the score goes up; if no words match at all all scores become zero as expected.
- Pandas DataFrames are very convenient. The search method returns a DataFrame, which means I can export ranked results straight to CSV or Excel with one call.

9. Files Submitted

No.	File / Folder	Description
1	<code>ir_project_code.py</code>	Main script with all 5 modules; runs top to bottom.
2	<code>tfidf_matrix_numerical.py</code>	Separate interactive tool: manual step-by-step TF-IDF calculation, good for exam practice.
3	<code>documents_dataset/</code>	Folder with 15 extra .txt files collected earlier in case a larger corpus was needed.
4	<code>generate_pdf.py</code>	The Python script I used to generate this report PDF.
5	<code>ir.py</code>	An older draft of the search engine code, kept for personal reference.
6	<code>IR_Project_Documentation.pdf</code>	This report itself in PDF form.

End of report. All numerical outputs printed here can be reproduced by running: `python ir_project_code.py`